BiblioFlow — Change request CR-BF-01

Report di analisi, progettazione e realizzazione

Ingegneria, Gestione ed Evoluzione del Software — A.A. 2025/2026

Gruppo

Mario Celzo — NF22500193

Renato Mancino — NF22500242

Alfonso Maria Ferrara — NF22500210

Repository: https://github.com/mariocelzo/biblioflow-app — codice verificato al commit `066f198`

Data: 13 ottobre 2026

Indice

1. Identificazione....4
2. Sintesi della change request....4
3. Processo di impact analysis....4
3.1 Metodo seguito....4
3.2 Evidenze AS-IS....5
3.3 Componenti impattati direttamente....5
3.4 Componenti impattati indirettamente — ripple effect....5
4. Progettazione TO-BE....6
4.1 Servizio di dominio unico....6
4.2 Atomicità e concorrenza....6
4.3 Modello della lista d'attesa....6
4.4 Identità, ruoli e ownership....7
4.5 Log, notifiche e aggiornamento client....7
4.6 Diagrammi....7
5. Realizzazione incrementale....7
5.1 Sequenza degli incrementi....7
5.2 Migrazioni dati....8
5.3 Cronologia Git sintetica....8
6. Criteri di accettazione ed evidenze....9
7. Strategia ed esiti di test....9
7.1 Deliverable....9
7.2 Risultati consolidati....9
8. Anomalie e limiti osservati....10
9. Valutazione finale....10
Appendice A — Analisi AS-IS e impact analysis....11
Componenti del sottosistema prenotazione — AS-IS....11
Stati di Prenotazione — AS-IS....14
Matrice ruoli-operazioni — AS-IS....15
Appendice B — Progettazione TO-BE....21
Componenti del sottosistema prenotazione — TO-BE....21

Stati di prenotazione e lista d’attesa — TO-BE 22
Sottosistema prenotazioni — confronto AS-IS / TO-BE 24
Appendice C — Verifica finale dei criteri di accettazione 27
Verifica finale dei criteri di accettazione — CR-BF-01 27

1. Identificazione

Campo	Valore
Progetto	BiblioFlow
Change request	CR-BF-01
Task documentale	BIB-65
Corso	Ingegneria, Gestione ed Evoluzione del Software — A.A. 2025/2026
Baseline immutabile	tag `baseline-pre-cr-bf-01`, commit `e0742bfca4c5b05a0223bbca71f9756af9f42596`
Versione finale analizzata	candidata del 2026-10-13: `main` @ `59a0d54` + PR #82 + PR #83 + chiusura finale
Esito dei criteri	6 criteri verificati su 6

2. Sintesi della change request

CR-BF-01 ha reso sicuro e consistente il processo di prenotazione dei posti studio. Nella baseline l'identità poteva provenire dal client, la verifica di disponibilità e la scrittura erano separate e non esisteva una lista d'attesa. La modifica introduce:

1. identità e ownership derivate dalla sessione autenticata;
2. validazione centralizzata degli intervalli;
3. creazione atomica con transazione e vincolo PostgreSQL;
4. lista d'attesa FIFO senza duplicati attivi;
5. promozione automatica e idempotente su cancellazione, no-show e scadenza;
6. log e notifiche persistenti correlati alle transizioni;
7. adeguamenti di API, amministrazione, interfacce e statistiche necessari a evitare regressioni.

L'intervento è evolutivo: amplia il flusso di prenotazione senza riprogettare l'intera applicazione, cambiare framework o modificare i processi di prestito e preparazione libri oltre ai controlli di regressione.

3. Processo di impact analysis

3.1 Metodo seguito

L'impact analysis è stata eseguita prima delle modifiche applicative e ha seguito un processo ripetibile:

1. **Congelamento della baseline.** Il tag `baseline-pre-cr-bf-01` è stato mantenuto immutabile per separare comportamento originale e TO-BE.
2. **Inventario statico.** Sono stati esaminati schema Prisma, route handler, import, chiamate HTTP, accessi al database, middleware, automazioni, notifiche, componenti UI e statistiche.
3. **Modellazione AS-IS.** Sono stati prodotti diagramma dei componenti, diagramma degli stati e matrice ruoli-operazioni.
4. **Caratterizzazione eseguibile.** I casi pre-modifica hanno fissato i comportamenti corretti e i difetti osservati prima di toccare il codice.
5. **Propagazione dell'impatto.** Per ogni componente direttamente modificato sono stati seguiti produttori, consumatori e dati condivisi, identificando i ripple effect.
6. **Realizzazione incrementale.** Dati e dominio hanno preceduto API, automazioni, interfacce e verifiche finali.

7. **Confronto post-modifica.** I casi sono stati classificati come aggiunti, modificati, invariati o eliminati e collegati ai criteri CA-01–CA-06.

3.2 Evidenze AS-IS

I documenti di analisi originali restano separati dal presente report; nella versione impaginata (PDF e Word) diagrammi e matrice sono riprodotti integralmente nell'Appendice A.

- diagramma componenti AS-IS;
- diagramma stati Prenotazione AS-IS;
- matrice ruoli-operazioni AS-IS;
- specifica dei test pre-modifica;
- report di esecuzione pre-modifica.

L'analisi ha evidenziato quattro cause principali:

- alcune route si fidavano di `userId` proveniente da payload o query string;
- creazione ed estensione duplicavano regole di validazione;
- controllo della disponibilità e creazione non costituivano un'operazione atomica e il database non impediva le sovrapposizioni;
- cancellazioni e no-show liberavano un posto senza alcun ciclo di coda.

3.3 Componenti impattati direttamente

Componente	Motivo dell'impatto	Risultato TO-BE
`prisma/schema.prisma` e migrazioni	Servivano entità, stati, indici e garanzie DB.	`ListaAttesa`, enum di coda, unicità attiva, indice FIFO e exclusion constraint sulle prenotazioni attive.
`src/lib/auth.ts`	L'identità non poteva restare autorevole nel payload.	Helper `requireUser`, `requireRole` e `assertOwnership` con errori 401/403/404 coerenti.
`src/lib/prenotazioni-service.ts`	Le regole erano duplicate tra endpoint.	Unico servizio di dominio per validazione, transazioni, coda e promozione.
`src/app/api/prenotazioni/**`	Le API dovevano usare sessione, dominio e contratti sicuri.	Ownership, validazione condivisa, creazione atomica, estensione coerente e API coda.

3.4 Componenti impattati indirettamente — ripple effect

Area	Dipendenza propagata	Adeguamento e rischio evitato
Automazioni e cron	No-show e scadenze liberano intervalli prenotabili.	Invocazione della promozione di dominio, lock e idempotenza; evitati doppi assegnamenti e notifiche duplicate.
Cancellazione amministrativa	La cancellazione staff modifica lo stesso stato usato dalla coda.	Promozione dopo il rilascio e feedback esplicito; evitato un posto libero con coda ferma.
Check-in e scanner	Una richiesta in attesa non equivale a una prenotazione.	Controlli su identità, ownership e stati ammessi; evitato il check-in di utenti non promossi.
Notifiche e log	Nuove transizioni richiedono nuovi tipi persistiti.	Enum estesi, riferimenti correlati e rendering con fallback; conservati i tipi storici.
Aggiornamento client	La disponibilità cambia anche per coda e automazioni.	Rilettura autenticata delle API e polling leggero delle notifiche; rimossi helper SSE scollegati e in-memory.
Mappa e vista mobile	Il conflitto non è più un vicolo cieco.	Ingresso in coda da entrambe le viste e

Area	Dipendenza propagata	Adeguamento e rischio evitato
		dal dialog di conflitto.
Middleware e route admin	I nuovi endpoint e i dati amministrativi devono rispettare RBAC.	Protezione delle route coda e chiusura dei varchi 401/403 individuati dalla verifica finale.
Statistiche	I nuovi eventi non devono alterare indicatori preesistenti.	Filtri e indicatore dedicato alla coda; conservata la distribuzione no-show.

Questa distinzione ha impedito di limitare la modifica ai soli file nominati dalla richiesta: la propagazione verso i consumatori è stata trattata come parte verificabile della manutenzione, non come lavoro accessorio.

4. Progettazione TO-BE

4.1 Servizio di dominio unico

`src/lib/prenotazioni-service.ts` è il punto unico delle regole di prenotazione e lista d'attesa. Non dipende da `Request` o `Response`, quindi gli endpoint traducono soltanto input, sessione ed errori HTTP. Il servizio controlla:

- data non passata, ordine degli orari e durata massima;
- apertura della sala e stato attivo/non in manutenzione del posto;
- sovrapposizioni sullo stesso posto;
- ingresso, annullamento e posizione FIFO della richiesta;
- promozione del primo utente eleggibile.

Gli errori hanno codice stabile e messaggio non tecnico. Creazione ed estensione non possono quindi divergere silenziosamente nell'interpretazione dello stesso intervallo.

4.2 Atomicità e concorrenza

La sola verifica applicativa non è sufficiente: due richieste possono leggere contemporaneamente lo stesso stato libero. La soluzione usa due livelli:

1. transazioni Prisma con isolamento `Serializable` per mantenere verifica e scrittura nella stessa unità atomica;
2. exclusion constraint PostgreSQL sull'intervallo delle prenotazioni attive, come ultima garanzia anche per scritture esterne all'applicazione.

Le violazioni di vincolo e i conflitti serializzabili vengono tradotti in un errore di dominio gestito. Il chiamante può proporre l'ingresso in lista d'attesa senza esporre dettagli del database.

4.3 Modello della lista d'attesa

`ListaAttesa` collega utente, posto, data e intervallo e usa gli stati `IN_ATTESA`, `PROMOSSA`, `SCADUTA` e `ANNULLATA`. L'ordine è deterministico: prima `createdAt`, poi `id` come criterio di spareggio. Il vincolo sui record attivi impedisce allo stesso utente di accodarsi due volte per lo stesso intervallo senza impedire la conservazione dello storico terminale.

La promozione seleziona il primo eleggibile, crea la prenotazione e aggiorna la richiesta nella stessa operazione transazionale. Chiamate ripetute non producono una seconda prenotazione.

4.4 Identità, ruoli e ownership

L'identità operativa deriva sempre dalla sessione. Il client può indicare una risorsa, ma non scegliere l'attore. Gli helper comuni distinguono:

- `401` quando manca una sessione valida;
- `403` quando il ruolo non permette l'operazione;
- `403` o `404` per una risorsa altrui secondo la policy documentata.

La configurazione NextAuth esistente non è stata riscritta: gli helper sono stati aggiunti sopra il contratto corrente e riusati dalle route.

4.5 Log, notifiche e aggiornamento client

Le transizioni di coda usano tipi espliciti (`CODA_INGRESSO`, `CODA_PROMOZIONE`, `CODA_SCADENZA`, `CODA_ANNULLATA`). Log e notifiche conservano i riferimenti a utente, posto, richiesta e prenotazione. La UI legge le notifiche dal database e aggiorna il badge periodicamente; la disponibilità viene richiesta al server per data e fascia. Il cron registra cosa ha processato e usa protezioni idempotenti, così una seconda esecuzione ravvicinata non cambia lo stato prodotto dalla prima.

4.6 Diagrammi

La fotografia AS-IS è disponibile nei documenti indicati nella sezione 3.2. I diagrammi TO-BE e il confronto affiancato sono stati prodotti dalla task correlata BIB-64, PR #49, integrata in `main`:

- diagramma dei componenti TO-BE, con servizio di dominio e lista d'attesa;
- diagramma delle transizioni TO-BE di `Prenotazione` e `ListaAttesa`;
- confronto AS-IS/TO-BE e sintesi dei ripple effect.

Nella versione impaginata del report (PDF e Word) i diagrammi TO-BE e il confronto sono riprodotti integralmente nell'Appendice B, accanto a quelli AS-IS dell'Appendice A, così che l'impact analysis sia leggibile senza aprire il repository.

5. Realizzazione incrementale

5.1 Sequenza degli incrementi

Fase	Incremento	Dipendenza soddisfatta
0	Regole Git, framework di test e pipeline CI.	Processo riproducibile prima delle modifiche.
1	Analisi AS-IS e test pre-modifica.	Baseline osservabile e difetti caratterizzati.
2	Schema, vincoli, autenticazione e servizio di dominio.	Fondamenta condivise per tutte le route.
3	API di prenotazione sicure e atomiche.	Esposizione HTTP del dominio senza identità client autorevole.
4	Automazioni, notifiche e tracciabilità.	Promozione su no-show/scadenza e audit persistente.
5	Amministrazione, middleware, UI e statistiche.	Chiusura dei ripple effect verso operatori e utenti.
6	Specifica post-modifica, esecuzione e regressione.	Verifica esplicita di CA-01–CA-06.
7	Diagrammi TO-BE e report finale.	Consolidamento dei deliverable del corso.

5.2 Migrazioni dati

Migrazione	Scopo
`20260102185756_migration1`	Schema iniziale della baseline (precedente alla CR).
`20260105115616_add_automation_types`	Tipi di automazione della baseline (precedente alla CR).
`20260829102939_lista_attesa`	Introduce entità, stato e indici della lista d'attesa.
`20260829131500_vincolo_anti_sovrapposizione_prenotazioni`	Impone la non sovrapposizione delle prenotazioni attive a livello PostgreSQL.
`20260829133500_estensione_enum_coda`	Estende, senza rinominare o rimuovere, gli enum di notifiche ed eventi.
`20260902113000_enable_rls_public_tables`	Abilita RLS sulle tabelle pubbliche come hardening della persistenza.
`20260902133000_test_pipeline_migrate_prod`	Smoke test della pipeline di deploy delle migrazioni (solo commento SQL).
`20260924120000_allinea_schema_richieste_preparazione`	Aggiunge allo storico oggetti presenti nello schema ma mai migrati: enum `StatoRichiesta`, preferenze di accessibilità di `User`, tabella `RichiestaPreparazione`.

Le migrazioni sono versionate e la pipeline di deploy usa Prisma in modo separato dalla build applicativa. I dati storici con stati terminali restano compatibili con i nuovi vincoli. Nella verifica del 13 ottobre 2026 le otto migrazioni, applicate a un PostgreSQL 16 vuoto, ricostruiscono uno schema già identico a `prisma/schema.prisma` (`prisma db push` non rileva differenze).

5.3 Cronologia Git sintetica

La cronologia completa resta in Git; qui sono riportati gli incrementi che ricostruiscono il percorso della CR:

Commit su `main`	Incremento
`8173151`, `c9695e5`	Convenzioni di lavoro e CI.
`7469969`, `92ee17d`, `ff50938`, `026eb5d`, `4de697b`, `6f59dc4`	Setup test, analisi e prove pre-modifica.
`b0e6138`	Fase 2 — dati e dominio, BIB-23–BIB-31.
`068e1be`	Fase 3 — API prenotazione sicure, BIB-32–BIB-39.
`01237b7`	Fase 4 — automazioni, notifiche e realtime, BIB-40–BIB-48.
`cb4b95c`, `ae26680`, `1c31817`, `9592aad`, `adc5e30`	Fase 5 — middleware, amministrazione, UI e statistiche.
`071d28a`, `bba0882`, `f4e8d82`, `e88977f`	Fase 6 — specifica, report post-modifica, regressione e verifica finale.
`daa090a`	Chiusura dei varchi CA-01 emersi durante la verifica finale.
`76ecb1e`	Hardening successivo all'audit API, dichiarato fuori dal perimetro funzionale CR-BF-01.
`59a0d54`	Rimozione della catena SSE mai collegata e correzioni finali su disponibilità e rate limiter.
PR #82	Collegamento dei rate limiter alle route critiche residue.
PR #83	Chiusura automatica dei `CHECK_IN` scaduti e disponibilità per fascia.
candidata 2026-10-13	Aggiornamento dipendenze di sicurezza, copertura delle restanti scritture, guardie `MANUTENZIONE`, documentazione e pacchetto.

Gli interventi di dipendenze, RLS e distribuzione delle migrazioni sono rimasti in commit separati per non confondere l'evoluzione funzionale con hardening e infrastruttura.

6. Criteri di accettazione ed evidenze

Criterio	Risultato	Evidenza principale
CA-01 — Accesso	PASS	Helper di sessione/ownership, suite auth e 36/36 casi admin dopo BIB-68.
CA-02 — Concorrenza	PASS	Vincolo DB, transazioni `Serializable` e test concorrenti PostgreSQL ripetuti.
CA-03 — Lista d'attesa	PASS	Unicità attiva, FIFO deterministico e flussi UI mappa/mobile.
CA-04 — Riassegnazione	PASS	Promozione su cancellazione/no-show/scadenza, lock e idempotenza.
CA-05 — Tracciabilità	PASS	Log, notifiche persistenti e correlazione delle transizioni di coda.
CA-06 — Regressione	PASS	20/20 casi invariati con esito identico alla baseline.

La decisione e le prove dettagliate sono in verifica finale dei criteri. Il documento conserva anche l'anomalia rilevata nella prima verifica di CA-01 e la successiva correzione, senza riscrivere retroattivamente l'evidenza.

7. Strategia ed esiti di test

7.1 Deliverable

Documento	Ruolo nel processo
Specifica pre-modifica	Definisce gli scenari originali e gli ID stabili.
Report pre-modifica	Registra l'esecuzione sulla baseline.
Specifica post-modifica	Classifica casi aggiunti, modificati, invariati ed eliminati.
Report parti modificate	Verifica funzionalità nuove e cambiate.
Report regressione	Confronta i 20 casi invariati con la baseline.
Verifica finale	Decide CA-01–CA-06 sulla versione integrata.

7.2 Risultati consolidati

- La specifica post-modifica registra 181 casi aggiunti, 14 modificati, 20 invariati e nessun caso eliminato alla prima esecuzione di Fase 6.
- Il report delle parti modificate ha prodotto 179 pass e 2 target di sicurezza inizialmente attesi come fallimenti.
- La candidata 2026-10-13 supera **758/758 test** in 102 file, inclusi i casi PostgreSQL, oltre a lint, typecheck e build di produzione.
- Il report di regressione ha prodotto **20/20 PASS**, senza differenze rispetto alla baseline per i casi invariati.
- La verifica focalizzata di concorrenza ha ripetuto cinque volte i due scenari PostgreSQL, per un totale di **10/10 PASS**.
- L'insieme focalizzato su CA-03, CA-04 e CA-05 ha prodotto **113/113 PASS**.

Le differenze su intervalli invertiti, ricomposizione `Date`/`Time`, check-in e payload utente sono correzioni volute e risultano classificate come casi modificati, non come regressioni.

8. Anomalie e limiti osservati

1. La prima verifica finale aveva rilevato tre varchi amministrativi relativi a 401/403. Sono stati documentati prima della correzione e chiusi da BIB-68.
2. Una singola esecuzione esplorativa della suite ha mostrato un errore non riproducibile dopo ricreazione del database; dieci esecuzioni complete consecutive e cinque esecuzioni della concorrenza sono risultate verdi.
3. La baseline PWA dichiarava due screenshot assenti e proteggeva alcuni asset prima del login. Il confronto Git dimostra che si tratta di limiti storici, non di regressioni della CR.
4. I test di caratterizzazione pre-modifica sono intenzionalmente separati dalla suite corrente: eseguirli indiscriminatamente sul codice TO-BE confonderebbe differenze volute e regressioni.
5. Il rate limiter è in memoria per singola istanza serverless: riduce gli abusi evidenti ma non è un limite globale. `REDIS_URL` resta predisposta per una futura implementazione distribuita.
6. L'audit di produzione (`npm audit --omit=dev`) è pulito; rimane un advisory high nella sola catena ESLint (`fast-glob`/`micromatch`/`braces`) senza fix compatibile pubblicato. Non entra nel bundle di produzione.

9. Valutazione finale

CR-BF-01 soddisfa i sei criteri di accettazione sulla versione verificata. La soluzione non affida la correttezza a un singolo livello: sessione e ownership proteggono l'accesso, il dominio centralizza le regole, le transazioni coordinano le scritture e il database garantisce l'assenza di sovrapposizioni anche nel caso limite. La lista d'attesa completa il flusso utente e i ripple effect sono coperti da automazioni, amministrazione, notifiche, UI e statistiche.

Il processo lascia una catena di tracciabilità completa:

```
change request
  -> impact analysis AS-IS
  -> test pre-modifica
  -> incrementi di dati, dominio, API e consumatori indiretti
  -> test post-modifica e regressione
  -> criteri CA-01–CA-06
  -> diagrammi e report TO-BE
```

Il report e i documenti collegati rendono verificabili sia il risultato tecnico sia il processo di manutenzione seguito per ottenerlo.

Appendice A — Analisi AS-IS e impact analysis

Componenti del sottosistema prenotazione — AS-IS

Scopo

Questo documento descrive le dipendenze effettive del sottosistema prenotazione nella baseline precedente alla CR-BF-01. È stato ricavato manualmente dagli import, dalle chiamate HTTP e dagli accessi Prisma presenti nel codice al commit `7469969` di `main`.

- Task: BIB-16 — Diagramma dei componenti del sottosistema prenotazione (AS-IS)
- Responsabile operativo: Renato Mancino, subentrato a Mario Celzo
- Baseline funzionale: tag `baseline-pre-cr-bf-01`
- Criteri di accettazione CR-BF-01: nessuno specifico; il documento alimenta l'impact analysis
- Documento complementare: BIB-17 — diagramma delle transizioni di stato
- Evidenza di regressione collegata: specifica dei test pre-modifica, integrata dalla task BIB-19

Relazione con l'analisi AS-IS esistente

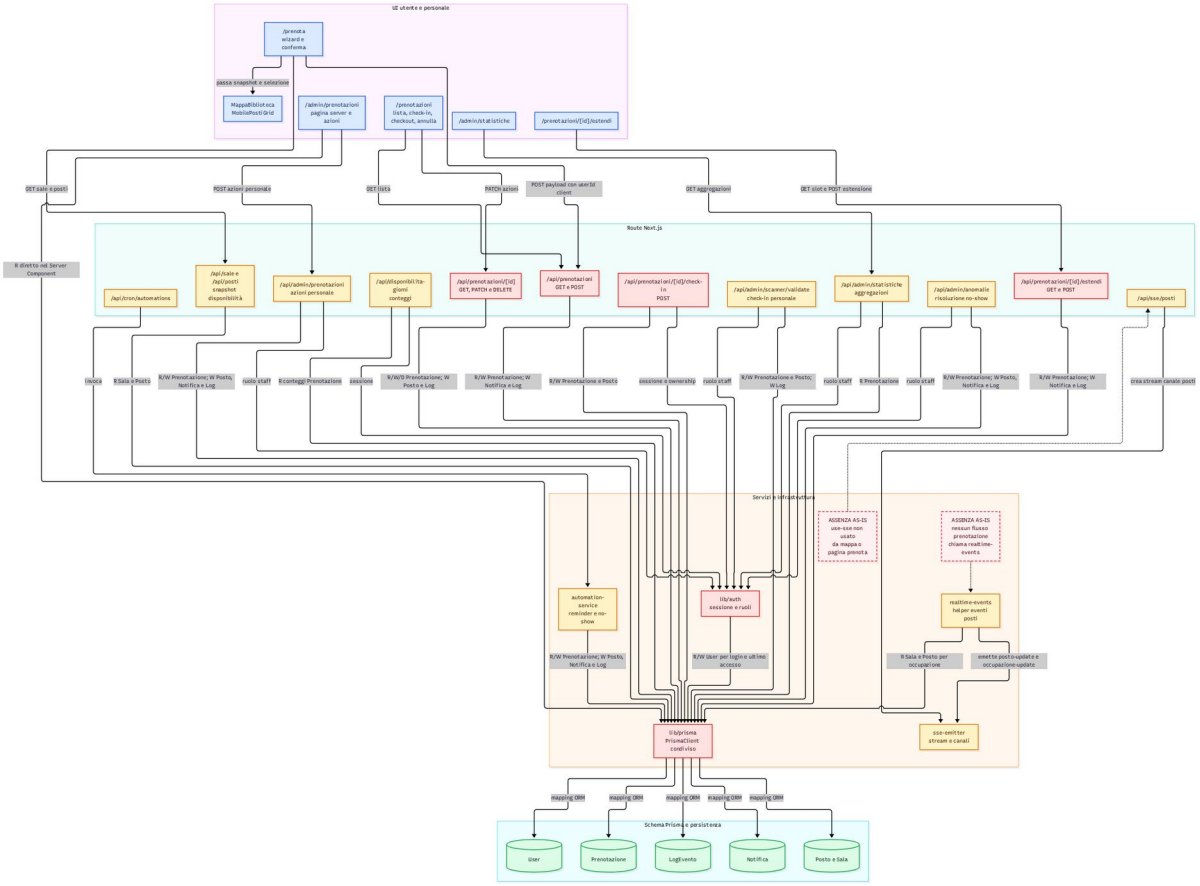
Questo elaborato non sostituisce il diagramma BIB-17 già presente nel repository. I due documenti osservano la stessa baseline da prospettive diverse:

Documento	Domanda a cui risponde	Contenuto riusato qui
BIB-17 — `prenotazione-stati-as-is.md`	Quali stati e transizioni sono effettivamente raggiungibili?	Stati, attori e anomalie delle transizioni sono usati come riferimento, senza duplicarne il diagramma.
BIB-19 — `spec-pre-modifica.md`	Quali comportamenti originali devono restare verificabili?	Automazioni, notifiche e SSE forniscono evidenze eseguibili per alcuni collegamenti descritti sotto.
BIB-16 — questo documento	Quali componenti partecipano al sottosistema e come dipendono tra loro?	Mappa UI → API → servizi → Prisma, con accessi R/W/D e punti di impatto della CR.

Legenda degli accessi

- **R**: lettura del modello `Prenotazione`.
- **W**: creazione o aggiornamento del modello `Prenotazione`.
- **D**: eliminazione fisica del record.
- Le frecce rosse tratteggiate descrivono collegamenti **assenti** nella baseline, non dipendenze attive.
- I nodi rossi sono punti di impatto diretto della CR; quelli gialli sono ripple effect da adeguare o riverificare.

Diagramma dei componenti



Accessi al modello `Prenotazione`

Componente	Accesso	Comportamento AS-IS	Effetto da considerare
`src/app/api/prenotazioni/route.ts`	R/W	Elenca, verifica sovrapposizioni e crea; riceve `userId` dal payload.	Punto diretto per identità da sessione, validazione centralizzata e atomicità.
`src/app/api/prenotazioni/[id]/route.ts`	R/W/D	Dettaglio, check-in/out, cancellazione ed eliminazione fisica.	Duplica transizioni e non applica sessione/ownership nella baseline.
`src/app/api/prenotazioni/[id]/check-in/route.ts`	R/W	Check-in autenticato con ownership e finestra temporale.	Va riallineato alle regole centralizzate e ai nuovi stati.
`src/app/api/prenotazioni/[id]/estendi/route.ts`	R/W	Calcola slot e conflitti con una logica propria, poi estende.	Ripple effect: può reintrodurre sovrapposizioni e divergenza di validazione.
`src/app/api/admin/prenotazioni/route.ts`	R/W	Annulla, modifica e forza check-in; aggiorna anche posto, log e notifica.	Ripple effect: la cancellazione staff dovrà attivare la promozione della coda.
`src/app/api/admin/anomalie/route.ts`	R/W	Individua e risolve prenotazioni senza check-in.	Ripple effect sul no-show e sulla promozione idempotente.
`src/app/api/admin/scanner/validate/route.ts`	R/W	Legge la prenotazione da QR e forza check-in.	Deve rifiutare stati non eleggibili, inclusa la futura attesa.
`src/lib/automation-service.ts`	R/W	Invia reminder; trasforma `CONFERMATA` in `NO_SHOW`	Ripple effect bloccante: il no-show dovrà promuovere

Componente	Accesso	Comportamento AS-IS	Effetto da considerare
		e libera il posto.	il primo eleggibile.
`src/app/api/admin/statistiche/route.ts`	R	Aggrega stati e trend.	I nuovi stati/eventi non devono alterare metriche e classificazioni.
`src/app/admin/prenotazioni/page.tsx`	R	Il Server Component legge Prisma direttamente per costruire la lista.	Accoppiamento UI-persistenza da riverificare dopo schema e stati nuovi.
`src/app/admin/page.tsx`, `src/app/admin/anomalie/page.tsx`	R	Conteggi e liste per dashboard/anomalie.	Nuovi stati possono cambiare conteggi, filtri e segnalazioni.
`src/app/api/disponibilita-giorni/route.ts`	R	Conta prenotazioni attive per giorno.	La disponibilità futura dovrà distinguere confermate e richieste in attesa.

Dipendenze senza accesso diretto a `Prenotazione`

Componente	Dipendenza effettiva	Osservazione AS-IS
`src/lib/auth.ts`	Usa `lib/prisma` per `User`; fornisce sessione e ruoli.	Solo check-in dedicato e route admin lo invocano. Le route generiche prenotazioni e l'estensione non derivano l'identità dalla sessione.
`src/app/prenota/page.tsx`	Chiama API sale, posti, profilo e prenotazioni.	Invia `session.user.id` nel JSON; non accede direttamente al DB.
`src/app/prenotazioni/page.tsx`	Chiama API lista e dettaglio per azioni utente.	Il filtro usa `utenteId`, mentre la route legge `userId`: dipendenza contrattuale incoerente da preservare come evidenza AS-IS.
`MappaBiblioteca` / `MobilePostiGrid`	Ricevono i posti via props da `/prenota`.	Non aprono uno stream SSE; rappresentano uno snapshot ottenuto da `/api/posti`.
`src/lib/sse-emitter.ts`	Mantiene stream e canali in memoria.	Non legge né scrive Prisma.
`src/lib/realtime-events.ts`	Importa `sse-emitter` e `lib/prisma` per statistiche di `Sala`/`Posto`.	Nessuna route o servizio di prenotazione chiama gli helper nella baseline; esistono ma sono scollegati dal flusso.
`src/hooks/use-sse.ts`	Espone `usePostiSSE()` verso `/api/sse/posti`.	Nessun componente UI lo importa nella baseline.

Punti di impatto diretto

1. **Identità e ownership.** `POST /api/prenotazioni` accetta `userId` dal client; le route generiche di dettaglio e l'estensione non usano `auth()`.
2. **Validazione duplicata.** Creazione, check-in ed estensione implementano controlli separati su stato, intervalli e sovrapposizioni.
3. **Atomicità assente.** La verifica della disponibilità e la creazione sono operazioni Prisma separate; non esiste un vincolo DB che impedisca due prenotazioni sovrapposte.
4. **Nuovo modello di attesa.** Schema Prisma, filtri e accessi al modello dovranno rappresentare una richiesta non ancora confermata senza renderla prenotabile o idonea al check-in.

Ripple effect da dimostrare nelle fasi successive

1. **Cancellazione staff e utente:** oltre a liberare il posto dovrà promuovere la coda una sola volta.
2. **No-show automatico e amministrativo:** `automation-service` e anomalie admin scrivono direttamente `Prenotazione` e `Posto`; devono condividere la stessa promozione idempotente.
3. **Estensione:** la ricerca conflitti locale deve usare la stessa regola atomica della creazione.

4. **Scanner e check-in:** i nuovi stati non confermati devono essere rifiutati.
5. **Statistiche e dashboard:** nuovi stati e log non devono inquinare trend, no-show e conteggi.
6. **Realtime:** la baseline contiene emettitore, endpoint e hook, ma non il collegamento completo. La CR dovrà collegare le transizioni a eventi e consumatori senza rompere la mappa esistente.
7. **Notifiche e log:** le route e le automazioni li scrivono insieme alle transizioni, spesso fuori da una transazione; i nuovi eventi devono restare coerenti con lo stato persistito.

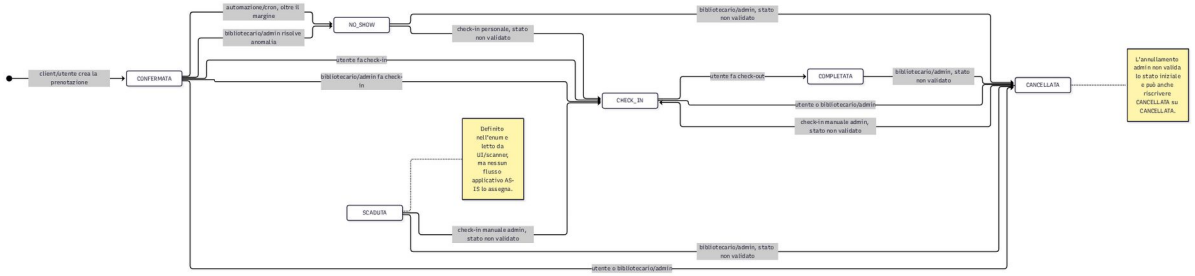
Confini del documento

Il diagramma rappresenta le dipendenze osservate, non l'architettura desiderata. Non introduce il futuro `prenotazioni-service`, la lista d'attesa o nuovi enum: questi appartengono ai diagrammi TO-BE e alle fasi di implementazione. Nessun file applicativo è stato modificato per produrre questa analisi.

Stati di `Prenotazione` — AS-IS

Questo documento descrive le transizioni effettivamente implementate nella baseline precedente alla CR-BF-01. Gli stati provengono dall'enum `StatoPrenotazione` in `prisma/schema.prisma`.

Il diagramma distingue il flusso ordinario dalle transizioni anomale che il codice attuale consente per assenza di un controllo sullo stato di partenza. Queste ultime non rappresentano il comportamento desiderato: sono parte dell'analisi AS-IS.



Evidenze nel codice

Stato iniziale	Stato finale	Attore	Evidenza AS-IS
nuovo record	`CONFERMATA`	client/utente	`POST /api/prenotazioni` crea esplicitamente la prenotazione come `CONFERMATA`; l'identità arriva attualmente dal payload.
`CONFERMATA`	`CHECK_IN`	utente	`PATCH /api/prenotazioni/[id]` con azione `check-in` e `POST /api/prenotazioni/[id]/check-in`; entrambi richiedono lo stato `CONFERMATA`.
`CONFERMATA`	`CHECK_IN`	bibliotecario/admin	`POST /api/admin/prenotazioni` con azione `CHECK_IN_MANUALE` e `POST /api/admin/scanner/validate`.
`CHECK_IN`	`COMPLETATA`	utente	`PATCH /api/prenotazioni/[i`

Stato iniziale	Stato finale	Attore	Evidenza AS-IS
			`d]` con azione `check-out`; l'azione richiede lo stato `CHECK_IN`.
`CONFERMATA`	`NO_SHOW`	automazione/cron	`releaseNoShowReservations()` in `src/lib/automation-service.ts`, invocata dall'endpoint cron, seleziona solo prenotazioni `CONFERMATA`.
`CONFERMATA`	`NO_SHOW`	bibliotecario/admin	Azione `ANNULLA_PRENOTAZIONI_SENZA_CHECKIN` in `src/app/api/admin/anomalie/route.ts`.
`CONFERMATA`, `CHECK_IN`	`CANCELLATA`	utente	`PATCH /api/prenotazioni/[id]` con azione `cancella`; gli altri stati vengono rifiutati.
qualunque stato	`CANCELLATA`	bibliotecario/admin	Le azioni `ANNULLA_SINGOLA` e `ANNULLA_MULTIPLE` in `src/app/api/admin/prenotazioni/route.ts` non controllano lo stato iniziale. Nel diagramma sono mostrate solo le transizioni che cambiano effettivamente valore.
`CANCELLATA`, `NO_SHOW`, `SCADUTA`	`CHECK_IN`	bibliotecario/admin	`CHECK_IN_MANUALE` rifiuta soltanto `CHECK_IN` e `COMPLETATA`; lo scanner rifiuta `CANCELLATA` e `SCADUTA`, ma non `NO_SHOW`. Queste transizioni sono quindi tecnicamente raggiungibili.

Stati terminali e stato non raggiungibile

Nel flusso ordinario `COMPLETATA`, `CANCELLATA` e `NO_SHOW` sono terminali. Le azioni amministrative prive di validazione possono tuttavia riaprire o sovrascrivere parte di questi stati, come indicato nel diagramma.

`SCADUTA` non ha alcuna transizione entrante implementata: compare nell'enum, nei tipi TypeScript, nei filtri dell'interfaccia amministrativa e nelle verifiche dello scanner, ma non viene mai scritto da una route, un servizio o il seed attuale.

La route `DELETE /api/prenotazioni/[id]` elimina fisicamente il record senza assegnare uno stato dell'enum; per questo non è rappresentata come transizione di stato.

Matrice ruoli-operazioni — AS-IS

Scopo e baseline

Questo documento fotografa i controlli di autenticazione e autorizzazione effettivamente implementati prima della CR-BF-01. L'analisi è stata ricavata manualmente dai route handler e da `src/middleware.ts` sulla baseline `main` al commit `7469969` (tag `baseline-pre-cr-bf-01`).

- Task: BIB-18 — Matrice ruoli-operazioni (AS-IS)

- Responsabile operativo: Renato Mancino, subentrato ad Alfonso Ferrara
- Ambito completo: `src/app/api/prenotazioni/**` e `src/app/api/admin/**`
- Inventario: 16 file route, 23 operazioni HTTP
- Criteri di accettazione CR-BF-01: nessuno specifico; la matrice prepara i test negativi di autorizzazione e la futura verifica di CA-01

Come leggere la matrice

Simbolo	Significato AS-IS
✅	Ruolo esplicitamente ammesso dopo `auth()` e controllo del ruolo nel route handler.
👤	Ammesso dopo `auth()` solo sulla prenotazione appartenente all'identità di sessione.
🔐	Ammesso da un handler che valida la sessione, ma non limita il ruolo.
⚠️	Nessun `auth()` nel handler e nessun controllo di ruolo; nel percorso HTTP ordinario resta soltanto il controllo superficiale del middleware.
❌	Ruolo esplicitamente respinto dal route handler con HTTP 403.

Il middleware intercetta tutte le API non pubbliche e restituisce 401 quando non trova un cookie `authjs.session-token` o `__Secure-authjs.session-token`. Non decodifica però il token e non verifica né sessione né ruolo: se il handler non chiama `auth()`, la sola presenza del cookie consente di raggiungere la logica applicativa. Per questo ⚠️ non equivale a un'autorizzazione verificata.

Area prenotazioni

Endpoint	Operazione	STUDENTE	BIBLIOTECARIO	ADMIN	Guardia effettiva	Identità o risorsa target
`/api/prenotazioni`	GET — elenco e filtri	⚠️	⚠️	⚠️	Nessun `auth()` nel handler.	`userId` opzionale dalla query; può indicare qualunque utente. Senza filtro restituisce tutte le prenotazioni.
`/api/prenotazioni`	POST — creazione	⚠️	⚠️	⚠️	Nessun `auth()` nel handler.	**`userId` arriva dal payload client** ed è usato per prenotazione, log e notifica.
`/api/prenotazioni/[id]`	GET — dettaglio	⚠️	⚠️	⚠️	Nessun `auth()` e nessuna ownership.	Prenotazione scelta tramite `id` nel path; include dati dell'utente e gli ultimi eventi.

Endpoint	Operazione	STUDENTE	BIBLIOTECARIO	ADMIN	Guardia effettiva	Identità o risorsa target
`/api/prenotazioni/[id]`	PATCH — check-in, check-out, cancellazione	⚠️	⚠️	⚠️	Nessun `auth()` e nessuna ownership.	Prenotazione scelta dal path. **`userId` opzionale dal payload** influenza il log, non autorizza l'azione.
`/api/prenotazioni/[id]`	DELETE — eliminazione fisica	⚠️	⚠️	⚠️	Nessun `auth()` e nessuna ownership.	Prenotazione scelta dal path; elimina anche i log collegati.
`/api/prenotazioni/[id]/check-in`	POST — check-in dedicato	👤	👤	👤	`auth()` e confronto `prenotazione.userId === session.user.id`; nessun vincolo di ruolo.	Identità dalla sessione; il `timestamp` operativo può arrivare dal payload client.
`/api/prenotazioni/[id]/estendi`	GET — slot di estensione	⚠️	⚠️	⚠️	Nessun `auth()` e nessuna ownership.	Prenotazione scelta dal path.
`/api/prenotazioni/[id]/estendi`	POST — estensione	⚠️	⚠️	⚠️	Nessun `auth()` e nessuna ownership.	Prenotazione scelta dal path; `nuovaOraFine` arriva dal payload client.

Risultato per l'area prenotazioni

Solo il check-in dedicato lega l'operazione alla sessione e verifica l'ownership. Tutte le altre operazioni possono essere eseguite da qualunque ruolo che superi il controllo di presenza del cookie del middleware; GET, PATCH, DELETE ed estensione non impediscono l'accesso a prenotazioni altrui.

Area admin

Endpoint	Operazione	STUDENTE	BIBLIOTECARIO	ADMIN	Guardia effettiva	Identità o risorsa target
`/api/admin/anomalie`	POST — risoluzione anomalie e azioni batch	❌	✅	✅	`auth()` più controllo ruolo staff.	Attore dalla sessione; azione e target (`userId` quando

Endpoint	Operazione	STUDENTE	BIBLIOTECARIO	ADMIN	Guardia effettiva	Identità o risorsa target
						richiesto) dal payload.
`/api/admin/posti/[id]`	GET — posto, prenotazioni e utenti collegati	🔐	🔐	🔐	`auth()` senza controllo ruolo.	Posto dal path; uno STUDENTE autenticato può leggere anche lo storico incluso.
`/api/admin/posti/[id]`	PATCH — cambio stato posto	❌	✅	✅	`auth()` più controllo ruolo staff.	Attore dalla sessione; posto dal path, stato e motivo dal payload.
`/api/admin/posti/[id]/dettagli`	GET — statistiche e storico posto	❌	✅	✅	`auth()` più controllo ruolo staff.	Posto dal path.
`/api/admin/prenotazioni`	POST — annulla singola/multiple, check-in manuale, modifica	❌	✅	✅	`auth()` più controllo ruolo staff, uguale per tutte le azioni.	Attore dalla sessione; azione, ID e nuovi dati dal payload.
`/api/admin/prestiti`	POST — restituisci, rinnova, sollecita	❌	✅	✅	`auth()` più controllo ruolo staff, uguale per tutte le azioni.	Attore dalla sessione; azione e ID dal payload.
`/api/admin/richieste`	GET — elenco richieste di preparazione	⚠️	⚠️	⚠️	**Nessun `auth()` e nessun controllo ruolo nel handler.**	Filtro `stato` dalla query; risposta con dati personali dell'utente.
`/api/admin/richieste`	PATCH — modifica stato e note richiesta	⚠️	⚠️	⚠️	**Nessun `auth()` e nessun controllo ruolo nel handler.**	ID, stato e note arrivano dal payload client.
`/api/admin/scanner/validate`	POST — check-in tramite QR	❌	✅	✅	`auth()` più controllo ruolo staff.	Attore dalla sessione; QR dal payload contiene prenotazione e utente

Endpoint	Operazione	STUDENTE	BIBLIOTECARIO	ADMIN	Guardia effettiva	Identità o risorsa target
						firmati.
`/api/admin/statistiche`	GET — aggregazioni	❌	✅	✅	`auth()` più controllo ruolo staff.	Tipo di statistica dalla query.
`/api/admin/utenti/[id]`	GET — dettaglio utente	❌	✅	✅	`auth()` più controllo ruolo staff.	Utente target dal path.
`/api/admin/utenti/[id]`	PATCH — attiva/disattiva utente	❌	❌	✅	`auth()` più controllo esclusivo `ADMIN`; vietata l'auto-disattivazione.	Attore dalla sessione; utente dal path e `attivo` dal payload.
`/api/admin/utenti/[id]/notifica`	POST — invia notifica	❌	✅	✅	`auth()` più controllo ruolo staff.	Attore dalla sessione; destinatario dal path e contenuto dal payload.
`/api/admin/utenti/[id]/profilo`	GET — profilo completo	❌	✅	✅	`auth()` più controllo ruolo staff.	Utente target dal path.
`/api/admin/utenti/[id]/storico`	GET — storico eventi	❌	✅	✅	`auth()` più controllo ruolo staff.	Utente target dal path.

Risultato per l'area admin

La maggior parte delle route applica correttamente la coppia `BIBLIOTECARIO`/`ADMIN`; la modifica dello stato di un utente è riservata ad `ADMIN`. Le eccezioni AS-IS sono:

1. `GET /api/admin/posti/[id]` valida la sessione ma non il ruolo e rende quindi disponibili a uno STUDENTE dati amministrativi, prenotazioni recenti e identità collegate.
2. `GET` e `PATCH /api/admin/richieste` non chiamano `auth()` e dipendono soltanto dalla presenza del cookie controllata dal middleware.

Origine dell'identità e decisioni di autorizzazione

Origine	Operazioni	Conseguenza AS-IS
Sessione verificata con ownership	Solo `POST /api/prenotazioni/[id]/check-in`.	Impedisce il check-in di una prenotazione altrui, indipendentemente dal ruolo.
Sessione verificata come attore staff	Route admin protette.	Il target resta normalmente nel path/payload, ma l'attore dei log deriva dalla sessione.
Payload client	`POST /api/prenotazioni`	Un client può creare per un altro utente o

Origine	Operazioni	Conseguenza AS-IS
	(`userId`), `PATCH /api/prenotazioni/[id]` (`userId` opzionale per il log).	attribuire il log a un'identità arbitraria.
Query string	`GET /api/prenotazioni?userId=...`	Un client può enumerare le prenotazioni di altri utenti o omettere il filtro per leggerle tutte.
Solo ID nel path, senza ownership	Dettaglio, aggiornamento, eliminazione ed estensione della prenotazione.	La conoscenza dell'ID è sufficiente per leggere o modificare la risorsa dopo il solo passaggio del middleware.
Cookie controllato solo per presenza	Handler marcati ⚠️.	Un cookie non vuoto raggiunge il handler; la validità della sessione non è verificata localmente.

Base per i test negativi di autorizzazione

Gli identificativi seguenti sono proposti come riferimenti stabili per le suite post-modifica. Descrivono i varchi AS-IS da chiudere, non test già implementati in questa task.

ID	Scenario negativo da verificare dopo la modifica	Esito richiesto dalla CR
`AUTH-NEG-001`	Richiesta protetta senza sessione.	HTTP 401.
`AUTH-NEG-002`	Cookie presente ma sessione/token non valido.	HTTP 401 prima dell'accesso ai dati.
`AUTH-NEG-003`	STUDENTE legge l'elenco senza filtro o usa `userId` altrui.	Accesso limitato alla propria identità o rifiuto secondo policy.
`AUTH-NEG-004`	STUDENTE crea una prenotazione con `userId` altrui nel payload.	Il payload non decide l'identità; viene usata la sessione.
`AUTH-NEG-005`	Utente legge, modifica, elimina o estende una prenotazione altrui.	HTTP 403/404 secondo policy.
`AUTH-NEG-006`	STUDENTE chiama `GET /api/admin/posti/[id]`.	HTTP 403.
`AUTH-NEG-007`	STUDENTE legge o aggiorna `/api/admin/richieste`.	HTTP 403.
`AUTH-NEG-008`	BIBLIOTECARIO tenta di attivare/disattivare un utente.	HTTP 403.
`AUTH-NEG-009`	Ruolo non staff chiama ciascuna route admin rimanente.	HTTP 403.

Tracciabilità e documenti collegati

- BIB-17 — diagramma delle transizioni di stato: descrive quali transizioni possono essere attivate dagli endpoint qui censiti.
- BIB-16 — diagramma dei componenti, PR #8: mostra le dipendenze UI → API → auth/servizi → Prisma.
- BIB-21 — test pre-modifica admin e UI, PR #7: fornisce la baseline di regressione dell'area amministrativa.

Confini del documento

La matrice descrive il comportamento osservato, non quello desiderato. Non modifica middleware, route, ruoli o policy e non introduce test eseguibili: serve come input documentale ai test negativi e agli interventi delle fasi successive.

Appendice B — Progettazione TO-BE

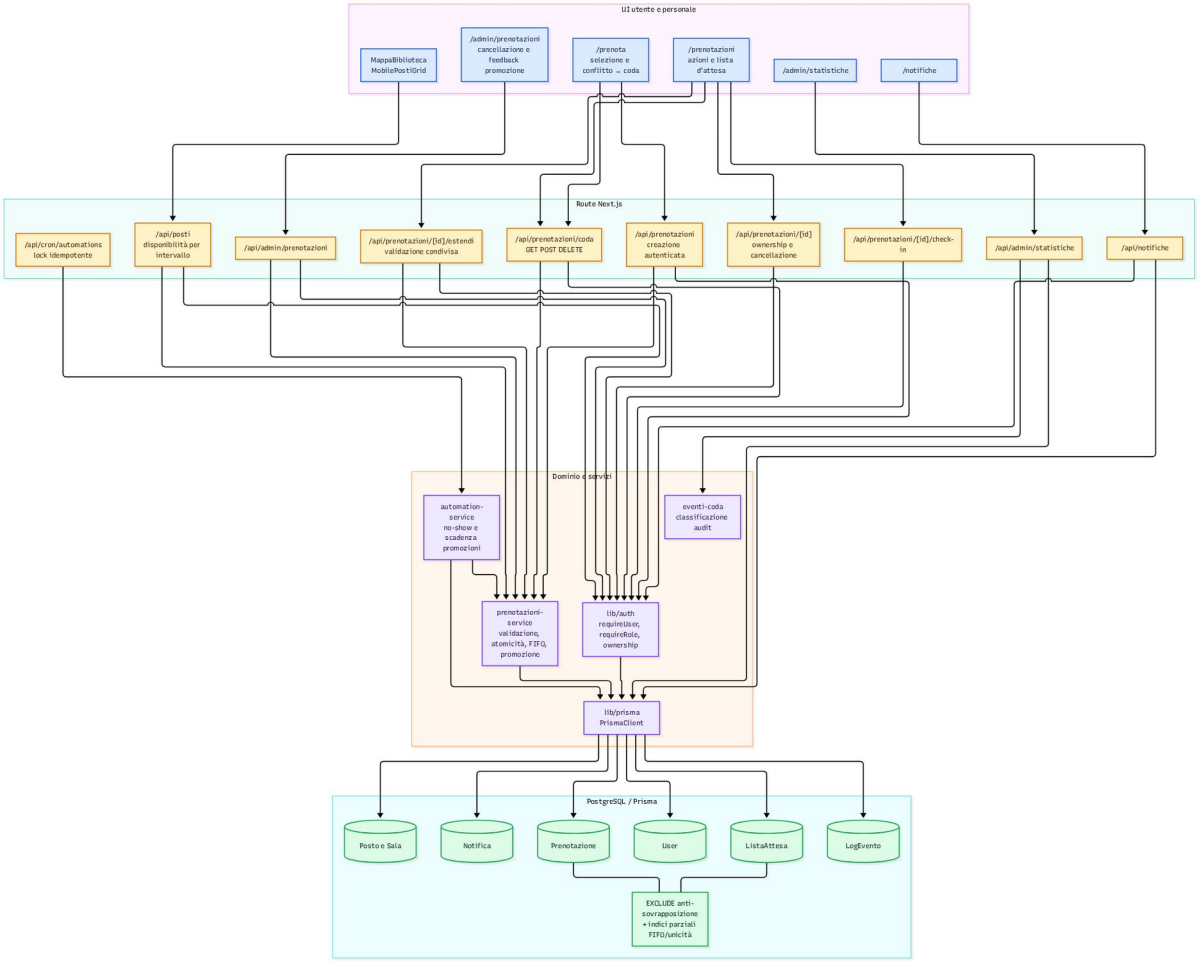
Componenti del sottosistema prenotazione — TO-BE

Scopo e riferimenti

Questa vista descrive l'architettura effettiva dopo CR-BF-01, verificata sul codice candidato alla consegna del 2026-10-13, basato su `main` alla baseline `59a0d54` e sull'integrazione delle PR #82/#83. Completa, senza sostituirla, l'analisi AS-IS dei componenti.

- Task: BIB-64
- Prerequisito: documenti e verifiche di Fase 6 integrati in `main`
- Vista complementare: stati TO-BE
- Confronto sintetico: AS-IS / TO-BE

Diagramma dei componenti



Responsabilità consolidate

Componente	Responsabilità TO-BE	Garanzia principale
`src/lib/auth.ts`	Identità, ruolo e ownership derivati dalla sessione	CA-01; niente identità autorevole dal payload
`src/lib/prenotazioni-service.ts`	Intervalli, conflitti, transazioni, coda FIFO e promozione	CA-02, CA-03 e CA-04 in un solo dominio
`Prenotazione` + migrazione SQL	Persistenza delle prenotazioni attive	Exclusion constraint su posto/data/intervallo

Componente	Responsabilità TO-BE	Garanzia principale
`ListaAttesa`	Richiesta distinta dalla prenotazione	Unicità parziale `IN_ATTESA` e ordine `createdAt, id`
Route `/api/prenotazioni/coda`	Ingresso, elenco con posizione e annullamento	Operazioni autenticate e ownership
`automation-service`	No-show, scadenza delle promozioni e nuova promozione	Effetti idempotenti e audit correlato
Route cron	Serializzazione dei run tramite advisory lock	Nessuna doppia elaborazione concorrente
Notifiche persistenti + polling	Aggiornamento personale e badge ogni 60 secondi	Nessuna connessione SSE in-memory su istanze serverless
Notifiche e `LogEvento`	Tracciabilità di ingresso, promozione, scadenza e annullamento	CA-05 e correlazione delle catene automatiche

Flussi principali

1. La route di creazione ricava l'utente dalla sessione e delega la scrittura atomica al servizio di dominio; il vincolo PostgreSQL resta l'ultima difesa.
2. In caso di intervallo occupato, la UI propone la coda e la route dedicata crea una `ListaAttesa` `IN_ATTESA`, con posizione FIFO calcolata dal dominio.
3. Cancellazione del personale o no-show liberano lo slot e invocano la stessa funzione `promuoviPrimoInCoda`; la promozione crea una `Prenotazione`, aggiorna la richiesta e registra `CODA_PROMOZIONE` nella medesima transazione.
4. Una promozione non confermata tramite check-in scade; automazione, notifica, audit e promozione del successivo chiudono il ciclo della coda.
5. La UI rilegge disponibilità e notifiche tramite API autenticate; i dettagli personali restano nelle notifiche persistenti del solo destinatario.

Confini

Il diagramma rappresenta dipendenze presenti nel codice finale. Non attribuisce a `ListaAttesa` lo stato di una prenotazione: le due entità mantengono cicli di vita separati e vengono correlate dall'audit `CODA_PROMOZIONE`.

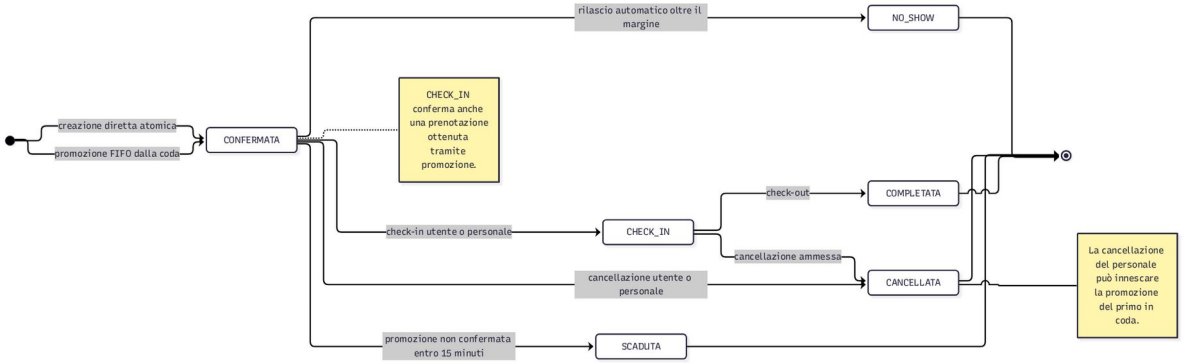
Stati di prenotazione e lista d'attesa — TO-BE

Scopo e convenzioni

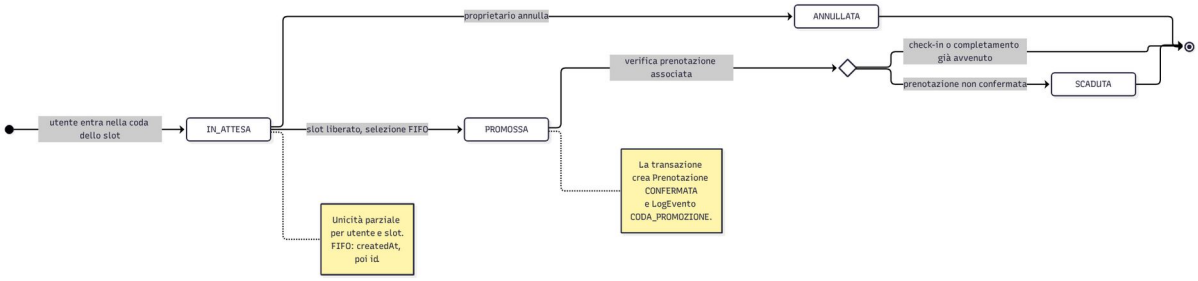
Il TO-BE distingue le due entità persistenti introdotte o modificate dalla CR-BF-01. `ListaAttesa` non è uno stato di `Prenotazione`: una prenotazione nasce solo dalla creazione diretta riuscita o da una promozione FIFO.

- Analisi precedente: stati Prenotazione AS-IS
- Architettura aggiornata: componenti TO-BE
- Confronto: AS-IS / TO-BE

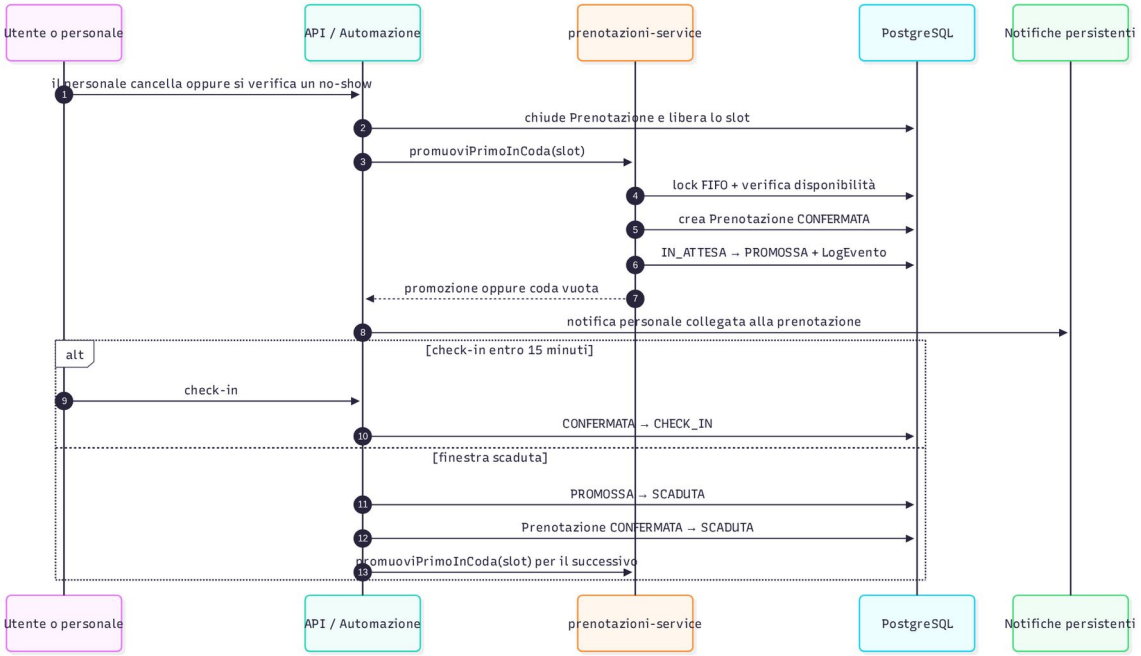
Ciclo della prenotazione



Ciclo della lista d'attesa



Coordinamento fra i due cicli



Transizioni e garanzie

Entità	Da	A	Innesco	Effetti osservabili
`ListaAttesa`	nuovo	`IN_ATTESA`	ingresso autenticato	`CODA_INGRESSO`, notifica, posizione FIFO
`ListaAttesa`	`IN_ATTESA`	`ANNULLATA`	proprietario	`CODA_ANNULLATA`
`ListaAttesa`	`IN_ATTESA`	`PROMOSSA`	cancellazione, no-show o scadenza precedente	nuova `Prenotazione`,

Entità	Da	A	Innesco	Effetti osservabili
				`CODA_PROMOZIONE`, notifica persistente
`ListaAttesa`	`PROMOSSA`	`SCADUTA`	nessun check-in entro la finestra	`CODA_SCADENZA`, rilascio e tentativo sul successivo
`Prenotazione`	nuovo	`CONFERMATA`	creazione o promozione	transazione Serializable e vincolo DB
`Prenotazione`	`CONFERMATA`	`CHECK_IN`	utente autorizzato/personale	posto occupato e audit
`Prenotazione`	`CHECK_IN`	`COMPLETATA`	check-out	chiusura ordinaria
`Prenotazione`	attiva	`CANCELLATA`	utente	chiusura; rilascio del posto se era in `CHECK_IN`
`Prenotazione`	attiva	`CANCELLATA`	personale	rilascio e possibile promozione
`Prenotazione`	`CONFERMATA`	`NO_SHOW`	cron	rilascio e possibile promozione
`Prenotazione`	`CONFERMATA` promossa	`SCADUTA`	mancata conferma	rilascio e possibile promozione successiva

Le transizioni automatiche usano guardie sullo stato atteso e transazioni Serializable. Un secondo run non ripete gli effetti già persistiti; il vincolo di esclusione impedisce comunque due prenotazioni attive sovrapposte.

Sottosistema prenotazioni — confronto AS-IS / TO-BE

Vista affiancata

Prospettiva	AS-IS	TO-BE
Componenti	Diagramma componenti AS-IS	Diagramma componenti TO-BE
Stati	Diagramma stati AS-IS	Diagramma stati TO-BE
Baseline	tag `baseline-pre-cr-bf-01`	candidata alla consegna del 2026-10-13, basata su `main` @ `59a0d54` e sulle PR #82/#83

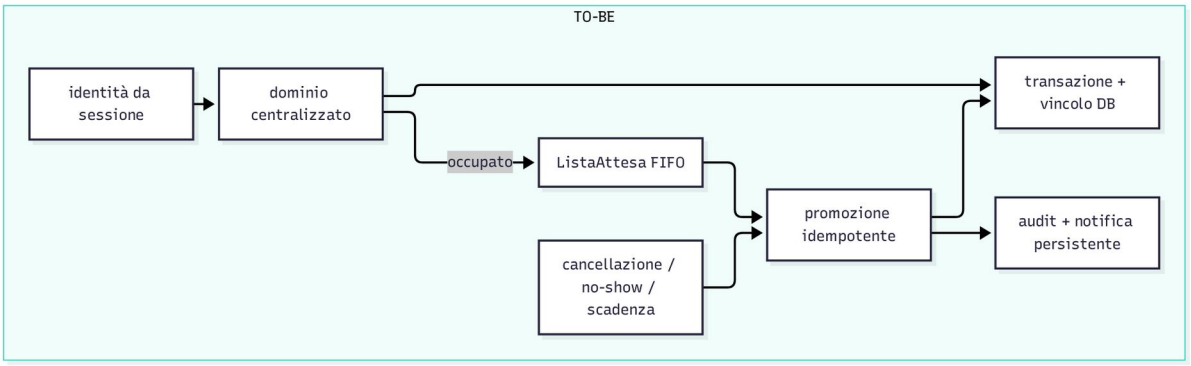
I diagrammi AS-IS restano immutati come fotografia della baseline. I documenti TO-BE rappresentano invece le dipendenze e le transizioni osservabili nel codice finale; questa pagina li mette in relazione senza duplicarne il dettaglio.

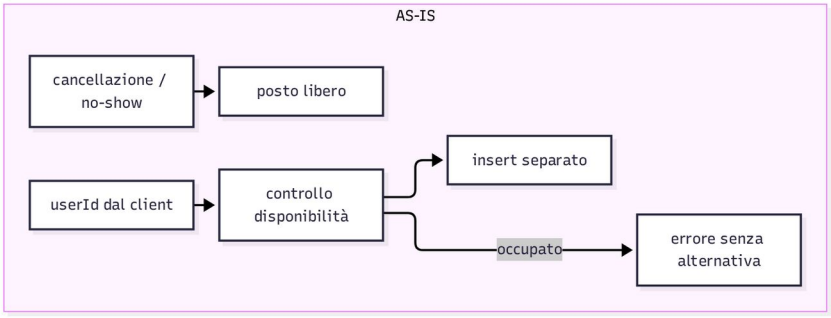
Differenze architetturali

Area	AS-IS	TO-BE	Evidenza finale
Identità	Alcune route si fidano del `userId` client	Identità e ownership dalla sessione	`requireUser`, `requireRole`, `assertOwnership`
Validazione	Regole replicate nelle route	Regole centralizzate	`src/lib/prenotazioni-service.ts`
Atomicità	Verifica e inserimento separati	Transazione Serializable più vincolo DB	`creaPrenotazioneAtomica` e migrazione `EXCLUDE`
Coda	Entità e API assenti	`ListaAttesa`, API dedicata, unicità e FIFO	schema, `/api/prenotazioni/coda`
Promozione	Nessuna riassegnazione	Un solo servizio idempotente condiviso	`promuoviPrimoInCoda`

Area	AS-IS	TO-BE	Evidenza finale
Cancellazione personale	Libera il posto soltanto	Libera e tenta la promozione	route `/api/admin/prenotazioni`
No-show	Libera il posto soltanto	Libera, promuove e traccia la catena	`automation-service`
Scadenza	`Prenotazione.SCADUTA` non raggiungibile	Promozione non confermata → scadenza e nuovo tentativo	`scadiPromozioniNonConfermate`
Cron	Run periodici senza coordinamento	Advisory lock e operazioni idempotenti	route `/api/cron/automations`
Audit	Eventi generici e non correlati	Eventi coda, attore e correlation ID	`LogEvento`, `eventi-coda`
Aggiornamento client	Helper SSE scollegati	Notifiche persistenti e polling leggero del badge; disponibilità riletta su data/fascia	`Notifica`, header, `/api/posti`
UI	Nessuna alternativa al conflitto	Ingresso/uscita coda e posizione	mappa, griglia mobile, pagina prenota
Statistiche	Ignorano la coda perché inesistente	Separano tempi e conversioni della coda	API e pagina statistiche

Evoluzione del flusso principale





Ripple effect chiusi

1. Creazione, estensione e controllo conflitti condividono la semantica degli intervalli; il database impedisce sovrapposizioni anche in concorrenza.
2. Check-in e ownership non accettano richieste in coda come prenotazioni.
3. Cancellazione del personale, no-show e scadenza convergono sulla stessa promozione, senza duplicarne l'algoritmo FIFO.
4. Cron e transizioni usano lock o guardie di stato per non duplicare promozioni, notifiche e log.

5. Le notifiche della coda sono persistenti e collegate alla prenotazione; il badge le rileva tramite polling senza mantenere connessioni SSE serverless.
6. Le statistiche distinguono gli eventi della coda dai conteggi storici.

Evidenza di verifica

La coerenza del TO-BE è supportata dai documenti di Fase 6:

- specifica dei test post-modifica;
- report delle parti modificate;
- report di regressione;
- verifica finale dei criteri.

Questi report costituiscono l'evidenza eseguibile; i diagrammi ne rappresentano la struttura e non introducono nuovi requisiti o comportamenti.

Appendice C — Verifica finale dei criteri di accettazione

Verifica finale dei criteri di accettazione — CR-BF-01

Identificazione

Campo	Valore
Task	BIB-63 — Verifica finale dei criteri di accettazione
Fase	6 — test post-modifica e regressione
Versione verificata	`main` (`adc5e30`) dopo il merge delle Fasi 4 e 5; aggiornamento CA-01 su branch `cr-bf-01-fix-bib-68-ca-01-admin-auth` (BIB-68)
Data	2026-09-04 CEST (prima verifica) — aggiornato 2026-09-04 CEST dopo BIB-68
Ambiente	Node.js 22.12.0, npm 10.9.0, Docker, PostgreSQL 16 Alpine
Esito complessivo	**CONFORME — 6 criteri verificati su 6**

Decisione

La change request è **accettabile**: tutti e sei i criteri sono verificati con evidenza automatizzata. Alla prima verifica (2026-09-04) **CA-01 non era soddisfatto**: tre richieste verso route admin non applicavano il codice 401/403 previsto (`GET /api/admin/posti/[id]`, `GET`/`PATCH /api/admin/richieste`). La task correttiva **BIB-68** ha chiuso i tre varchi con guardia "sessione + ruolo staff" eseguita prima di leggere parametri/body o toccare Prisma, e ha promosso i target `TC-BIB57-051`/`054` da `it.fails` a test ordinari verdi (più `TC-BIB57-050/052/053/055..060` a copertura del contratto 401/403/422). CA-02, CA-03, CA-04, CA-05 e CA-06 restano verificati come nella prima passata.

Questo documento registra sia l'esito originale (sezione "Prima verifica — CA-01 non conforme", conservata per tracciabilità) sia l'aggiornamento dopo BIB-68.

Matrice finale

Criterio	Esito	Evidenza principale	Decisione
CA-01 — accesso	**PASS** (dopo BIB-68)	`tests/integration/autorizzazione-admin.test.ts`: 36/36 passati, 0 `expected fail`; `TC-BIB57-050/051/052/053/054/055` provano la chiusura dei 3 varchi, `TC-BIB57-056/057/058/059/060` coprono staff/validazione	Verificato
CA-02 — concorrenza	**PASS**	suite PostgreSQL isolata ripetuta 5 volte: 10/10 casi passati	Verificato
CA-03 — lista d'attesa	**PASS**	vincolo DB, ordinamento FIFO/id, middleware e flussi UI nella selezione CA-03–05: 113/113	Verificato
CA-04 — riassegnazione	**PASS**	promozione, lock, idempotenza e ciclo end-to-end nella selezione CA-	Verificato

Criterio	Esito	Evidenza principale	Decisione
		03–05: 113/113	
CA-05 — tracciabilità	**PASS**	log, notifiche, correlazione e realtime nella selezione CA-03–05: 113/113	Verificato
CA-06 — regressione	**PASS**	BIB-62: 20/20 casi invariati passati, identici alla baseline	Verificato

Gate tecnici

Controllo	Esito	Dettaglio
`npm run lint`	PASS	0 errori, 24 warning preesistenti
`npx tsc --noEmit`	PASS	0 errori
`npm test` (dopo BIB-68)	**PASS pieno**	21/21 file; **187 passati, 0 `expected fail`** — i due target `it.fails` sono ora `it()` verdi
`npm run build`	PASS	Next.js 16; 43 pagine statiche; soli warning di deprecazione Sentry
`npm audit --omit=dev --audit-level=high`	PASS	0 vulnerabilità di produzione
`npm audit --audit-level=high`	WARN	5 voci high riconducibili a un solo advisory nella catena ESLint di sviluppo; il fix proposto richiede un downgrade incompatibile

Alla prima verifica il processo Vitest terminava comunque con codice 0 perché i due target di sicurezza erano espressi con `it.fails` — non rendeva CA-01 conforme, perché `it.fails` prova solo che il comportamento desiderato non è ancora implementato. Dopo BIB-68 quei due target sono `it()` ordinari e passano per il motivo giusto: la guardia è implementata.

CA-01 — accesso: verificato (dopo BIB-68)

Prima verifica (BIB-63, 2026-09-04) — non conforme

La selezione focalizzata aveva prodotto:

```
Test Files  4 passed (4)
Tests       45 passed | 2 expected fail (47)
```

I casi che descrivevano lo stato reale e i corrispondenti target erano:

Caso	Richiesta	Atteso	Osservato
`TC-BIB57-050` / target `051`	STUDENTE, `GET /api/admin/posti/[id]`	403	la logica handler viene eseguita; 404 o 200, mai 403
`TC-BIB57-052`	anonimo, `GET /api/admin/richieste`	401	200
`TC-BIB57-053`	anonimo, `PATCH /api/admin/richieste`	401	200 e mutazione eseguita
target `TC-BIB57-054`	STUDENTE, `GET /api/admin/richieste`	403	nessuna guardia di autenticazione/ruolo

Interventi individuati:

1. applicare una guardia staff a `GET /api/admin/posti/[id]`;
2. applicare autenticazione e ruolo staff a `GET` e `PATCH /api/admin/richieste`;

3. convertire `TC-BIB57-051` e `TC-BIB57-054` da `it.fails` a test ordinari;
4. rieseguire suite CA-01 e gate completi.

La correzione non era stata effettuata in BIB-63 per rispettare sia il perimetro di una task di verifica sia la responsabilità dell'area admin/UI.

Correzione (BIB-68) e verifica finale — conforme

BIB-68 ha applicato la guardia "sessione + ruolo staff" — eseguita **prima** di leggere parametri/body o interrogare Prisma — a `GET /api/admin/posti/[id]` e a `GET`/`PATCH /api/admin/richieste`; nel `PATCH` ha aggiunto anche la validazione di `stato` (contro l'enum Prisma `StatoRichiesta`) e di `note` (≤ 500 caratteri), entrambe 422 se non valide.

Matrice 401/403 verificata (`tests/integration/autorizzazione-admin.test.ts`, blocco 05x):

Route	Anonimo	STUDENTE	BIBLIOTECARIO/ADMIN
`GET /api/admin/posti/[id]`	401 (già coperto, `TC-BIB57-007`)	**403** (`TC-BIB57-050/051`)	consentito (`TC-BIB57-060`)
`GET /api/admin/richieste`	**401** (`TC-BIB57-052`)	**403** (`TC-BIB57-054`)	consentito, 200 (`TC-BIB57-056`)
`PATCH /api/admin/richieste`	**401** (`TC-BIB57-053`)	**403** (`TC-BIB57-055`)	consentito, muta (`TC-BIB57-057`)

In tutti i casi 401/403 nessuna query o mutazione Prisma viene invocata (asserito esplicitamente in ciascun test). Aggiunta anche la validazione del `PATCH`: `stato` fuori enum → 422 (`TC-BIB57-058`), `note` > 500 caratteri → 422 (`TC-BIB57-059`), nessuno dei due muta.

Esito della suite focalizzata dopo la correzione:

```
Test Files  1 passed (1)
Tests       36 passed (36)
```

`TC-BIB57-051` e `TC-BIB57-054` non sono più `it.fails`: sono `it()` ordinari e verdi. CA-01 è quindi **verificato** secondo lo stesso criterio applicato agli altri cinque.

CA-02 — concorrenza: verificato

`tests/integration/concorrenza.test.ts` è stato eseguito cinque volte in sequenza contro PostgreSQL reale. Ogni esecuzione comprende:

- due richieste simultanee ripetute internamente su tre round;
- N richieste simultanee con controllo dell'unico vincitore e assenza di record fantasma.

Esito aggregato:

```
RUN 1..5: exit 0
Test Files  1 passed (1) per esecuzione
Tests       2 passed (2) per esecuzione
Totale      10/10 passati
```

I messaggi Prisma relativi a write conflict, deadlock o violazione del vincolo di esclusione sono attesi: rappresentano le richieste perdenti che il dominio traduce in conflitto, non fallimenti del test.

CA-03, CA-04 e CA-05 — verificati

Una selezione di dieci file focalizzati su lista d'attesa, dominio, riassegnazione, automazioni, notifiche e realtime ha prodotto:

```
Test Files  10 passed (10)
Tests       113 passed (113)
```

Le evidenze includono:

- unicità DB della richiesta in coda e nessun duplicato;
- ordinamento deterministico per `createdAt` con `id` come tie-breaker;
- promozione transazionale senza sovrapposizioni;
- idempotenza di lock cron, scadenza e promozione;
- log correlati e notifiche per ingresso, uscita, promozione e scadenza;
- propagazione realtime ai soli canali previsti.

CA-06 — regressione: verificato

Il report BIB-62 confronta tutti i casi invariati con BIB-22:

```
Baseline        20/20 PASS
Post-modifica   20/20 PASS
Differenze      0
```

Le differenze del check-in sono miglioramenti intenzionali classificati come **MODIFICATI**, mentre i limiti PWA osservati esistevano già nella baseline. Non risultano regressioni introdotte dalla CR-BF-01 nel perimetro definito.

Stabilità della suite completa

Dopo l'esecuzione di diversi sottoinsiemi sullo stesso database, una prima esecuzione completa ha riportato un singolo test intermittente. L'output diagnostico non ha permesso di attribuirlo a un criterio. L'anomalia non si è riprodotta:

- 10 riesecuzioni complete consecutive: tutte verdi;
- ultima esecuzione dopo ricreazione del container, 7/7 migrazioni e fixture pulita: 179 passati, 2 `expected fail`, 0 fallimenti inattesi;
- suite concorrenza isolata: 5/5 esecuzioni verdi.

Il dato è registrato per trasparenza come probabile contaminazione dello stato di test durante la sessione esplorativa. Non c'è al momento un fallimento riproducibile da aprire come regressione; se ricompare in CI va conservato il log completo e aperta una task di stabilizzazione del test database.

Stato consigliato

- BIB-63: **6/6 — pronta per la revisione**, dopo il merge della PR correttiva di BIB-68.
- BIB-68: correzione implementata e verificata (401/403/422 su tutte e tre le route, 36/36 test focalizzati, 187/187 sulla suite completa); in revisione.
- BIB-7 (epic Fase 6): chiudibile dopo il merge della PR di BIB-68.
- BIB-60, BIB-61 e BIB-62: documenti disponibili nelle rispettive PR e pagine Confluence, invariati da questo aggiornamento.

Cronologia

Data	Evento
2026-09-04	Prima verifica (BIB-63): 5/6, CA-01 bloccato da 3 varchi sulle route admin.
2026-09-04	BIB-68: guardie applicate, `TC-BIB57-051/054` promossi da `it.fails` a `it()` verdi, 6 nuovi casi (`055-060`) a copertura di ruoli/validazione. Verifica finale aggiornata a **6/6**.